

FECAP — Centro Universitário Álvares Penteado

Cloud Native e Linux

Containerização, Deploy e Automação Bash

Projeto Interdisciplinar — 3º ADS 2026

Projeto: Maya Fisioterapia/RPG — Clínica Maya Yoshiko Yamamoto

Disciplina: Sistemas Operacionais e Arquiteturas Cloud Native

Aluno: Nelson Reis

Validação técnica: 11/05/2026 — ambiente local containerizado

Evidências: 25 prints em Imagens/cloud-native/

Entrega 2 — 4,0 pontos

Índice

1. Visão Geral e Artefatos
2. Docker — Dockerfile da API e do Banco
3. Docker Compose — Orquestração, Volume e Rede
4. Variáveis de Ambiente
5. Healthchecks
6. Scripts Shell/Bash (5 scripts)
7. Matriz de Aderência ao PI
8. Conceitos Linux/Bash Demonstrados
9. Ambiente Tradicional vs. Containerizado
10. Teste de Carga k6 — Resultado Real
11. Evidências Visuais — 25 prints

Relatório Cloud Native e Linux — Entrega 2

Projeto: Maya Fisioterapia/RPG — Backend/API

Aluno responsável (escopo individual): Programação Mobile + Cloud Native

Validação técnica: 11/05/2026 em ambiente local containerizado (Docker Desktop, Windows 11 + WSL/Git Bash)

Evidências visuais: `imagens-maya/cloud-native/`

Documento principal da entrega individual de Cloud Native/Infraestrutura com Linux. Cobre Dockerfile, docker-compose, scripts Shell/Bash, healthchecks, persistência por volume, variáveis de ambiente, deploy automatizado e teste de carga com k6. Cada item abaixo aponta para um print real anexado em `imagens-maya/cloud-native/`.

1. Visão geral

A solução foi organizada para funcionar em ambiente local containerizado, com backend, banco de dados e automação por scripts Shell/Bash.

O objetivo é demonstrar uma arquitetura replicável, com persistência, parâmetros de ambiente e separação entre aplicação e infraestrutura.

Caminhos reais dos artefatos

Os arquivos da entrega de Cloud Native estão nesta pasta (`Documentos/Entrega2/Sistemas Operacionais e Arquiteturas Cloud Native/`). O ambiente de validação local utilizou a cópia de trabalho do backend em `C:\Users\nelso\OneDrive\Nelson\Desktop\Workspace\maya-rpg-api`, motivo pelo qual os prints do Docker exibem esse caminho. Os artefatos versionados nesta pasta são os mesmos validados nos prints.

Artefato	Caminho versionado
Dockerfile da API	<code>./Dockerfile</code>
Dockerfile do banco	<code>./Dockerfile.db</code>
Compose	<code>./docker-compose.yml</code>
Variáveis de ambiente	<code>./.env.example</code>
Scripts Shell/Bash	<code>./scripts/*.sh</code>
Migrações SQL	<code>./scripts/migrations/*.sql</code>

2. Docker no backend

2.1 Dockerfile da API

Build multi-stage Node.js 20-alpine com etapa de build separada do runtime, usuário não-root (`appuser`), `curl` instalado para healthcheck e diretórios `/app/uploads` e `/app/logs` com permissão correta para o usuário do container.

Prints relacionados:

- `07-arquivos-docker.png` — listagem dos três arquivos (`Dockerfile`, `Dockerfile.db`, `docker-compose.yml`) no diretório de trabalho.
- `08-docker-compose-build.png` — `docker compose build` finalizando com `Image maya-rpg-api-api Built`.

2.2 Dockerfile.db

Base `postgres:16-alpine`, variáveis `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD` definidas como defaults (sobrescritas pelo Compose/`.env` em runtime), porta 5432 exposta. Mantido como artefato acadêmico exigido pelo PI (a Entrega 2 pede explicitamente "Dockerfile para o banco de dados (PostgreSQL/MySQL)").

Prints relacionados:

- `08-docker-compose-build.png` — `Image maya-rpg-db:latest Built`.
- `14-volume-ls-pg-isready.png` — `docker compose exec db pg_isready -U maya_user -d maya_rpg` → `accepting connections`.

3. Docker Compose

O `docker-compose.yml` organiza dois serviços (`api`, `db`) na rede interna `maya-network`, com `depends_on: db.condition: service_healthy`, volume nomeado `pg_data` para o PostgreSQL, bind mounts `./uploads:/app/uploads` e `./logs:/app/logs`, healthchecks para ambos os serviços e variáveis de ambiente injetadas a partir do `.env`.

Prints relacionados:

- `05-docker-compose-config-api.png` — bloco resolvido do serviço `api` (build, healthcheck `curl -f /api/docs`, ports `3000:3000`, bind mounts).
- `06-docker-compose-config-db.png` — bloco resolvido do serviço `db` (healthcheck `pg_isready`, ports `5432:5432`, volume `pg_data:/var/lib/postgresql/data`, rede `maya-network`).
- `09-docker-compose-up-healthy.png` — `docker compose up -d` finalizando com `Container maya-rpg-db Healthy` e `Container maya-rpg-api Started`.

3.1 Volume persistente

`pg_data:/var/lib/postgresql/data` garante a persistência dos dados do PostgreSQL entre reinicializações. Evidenciado em `14-volume-ls-pg-isready.png` (volume `maya-rpg-api_pg_data` listado por `docker volume ls`).

3.2 Volumes de uploads e logs

Bind mounts `./uploads` e `./logs` separam dados gerados em runtime do código-fonte. Evidenciado em `15-uploads-logs.png` (`docker compose exec api ls -la /app/uploads` e `/app/logs`).

4. Variáveis de ambiente

O `.env.example` documenta os campos esperados (banco, JWT, CORS, porta, SMTP, FCM, Mercado Pago, deploy). O `.env` real fica fora do Git (referenciado no `.gitignore`).

Boas práticas adotadas:

- `.env` não é versionado;
- `.env.example` serve como modelo, sem credenciais reais;
- cada ambiente ajusta suas variáveis sem alterar o código;
- durante execução via Docker Compose, `DB_HOST=db` (nome do serviço na rede interna).

Print relacionado:

- `01-env-configurado.png` — `.env` local preenchido para a demonstração (DB, JWT, CORS, PORT, SMTP, FCM, MP_*).

5. Healthchecks

Serviço	Healthcheck	Evidência
<code>api</code>	<code>curl -f http://localhost:3000/api/docs</code>	<code>09-docker-compose-up-healthy.png</code> , <code>10-docker-compose-ps.png</code>
<code>db</code>	<code>pg_isready -U \$DB_USER -d \$DB_NAME</code>	<code>10-docker-compose-ps.png</code> , <code>14-volume-ls-pg-isready.png</code>

A coluna `STATUS` em `10-docker-compose-ps.png` mostra `Up X seconds (healthy)` em ambos os containers.

6. Scripts Shell e Bash

Cinco scripts mantidos em `./scripts/`. A função de cada um e os prints que evidenciam execução real:

Script	Função	Evidência
<code>setup_env.sh</code>	Verificação automatizada das dependências do ambiente (Docker, Docker Compose, Node 20+, npm, <code>.env</code> , <code>node_modules</code>). Não instala dependências — relata o que está faltando e orienta o passo manual.	(Sem print dedicado — execução é silenciosa quando tudo está OK; o efeito prático aparece no fluxo do <code>deploy.sh</code> .)

Script	Função	Evidência
<code>monitor_system.sh</code>	Coleta métricas (CPU via <code>/proc/stat</code> , memória via <code>/proc/meminfo</code> , disco via <code>df</code> , containers via <code>docker ps</code>) e grava em <code>logs/monitor/metrics_*.log</code> .	<code>18-monitor-system-sh.png</code>
<code>backup_db.sh</code>	<code>pg_dump</code> do banco via <code>docker compose exec -T db</code> , salva em <code>backups/backup_YYYYMMDD_HHMMSS.sql</code> e aplica retenção em dias.	<code>16-backup-db-sh.png</code> (execução), <code>17-backup-arquivo-gerado.png</code> (arquivo <code>backup_20260511_125410.sql</code> listado em <code>backups/</code>)
<code>manage_services.sh</code>	<code>up</code> , <code>down</code> , <code>status</code> , <code>logs</code> , <code>restart</code> e modo <code>watch</code> (restart automático em falha).	<code>19-manage-services-sh.png</code> (<code>status</code> + <code>restart</code> completo com <code>API esta respondendo</code>)
<code>deploy.sh</code>	Deploy automatizado: verifica dependências → salva tag de rollback → <code>docker compose build --no-cache</code> → backup pré-deploy → <code>compose down</code> → aplica migrações SQL → <code>compose up -d</code> → healthcheck com timeout. Suporta <code>--rollback</code> .	<code>20-deploy-sh-inicio.png</code> a <code>24-deploy-sh-healthy.png</code> (5 prints sequenciais cobrindo o deploy completo)

6.1 Importante: escopo do `setup_env.sh`

O PI menciona "automatizar instalação de dependências (Java, Android SDK, ferramentas de build)" como exemplo da Entrega 1 (Infraestrutura e Automação com Linux). Na Entrega 2, o foco passou para containerização. O `setup_env.sh` mantido nesta entrega tem escopo de **verificação automatizada** (não instalação): confirma que Docker, Docker Compose, Node ≥ 20, npm, `.env` e `node_modules` estão presentes, e orienta o passo manual quando algum item falta. Esta decisão é deliberada — instalar Java/Android SDK exige privilégios e gerenciadores diferentes por SO (apt, brew, choco), o que tornaria o script frágil e não-reproduzível. O ciclo prático de bootstrap é coberto pelo Compose (`docker compose up --build -d`) e pelo `deploy.sh`.

7. Matriz de aderência ao PDF (Entrega 2)

Exigência da Entrega 2	Artefato	Evidência (print)	Aderência
Containerizar a API REST	<code>./Dockerfile</code>	<code>07-arquivos-docker.png</code> , <code>08-docker-compose-build.png</code>	Atendido

Exigência da Entrega 2	Artefato	Evidência (print)	Aderência
Preparar banco em container	<code>./Dockerfile.db</code> + serviço <code>db</code> no Compose	<code>06-docker-compose-config-db.png</code> , <code>08-docker-compose-build.png</code>	Atendido
Orquestrar API + BD	<code>./docker-compose.yml</code>	<code>05-docker-compose-config-api.png</code> , <code>06-docker-compose-config-db.png</code> , <code>09-docker-compose-up-healthy.png</code> , <code>10-docker-compose-ps.png</code>	Atendido
Volume persistente	<code>pg_data</code> nomeado	<code>14-volume-ls-pg-isready.png</code>	Atendido
Variáveis de ambiente	<code>./env.example</code> + bloco <code>environment</code> do Compose	<code>01-env-configurado.png</code> , <code>05-docker-compose-config-api.png</code>	Atendido
Script de deploy automatizado	<code>./scripts/deploy.sh</code>	<code>20-deploy-sh-inicio.png</code> → <code>24-deploy-sh-healthy.png</code>	Atendido
Documentação de build/execução	Seção 9 deste documento + <code>docs/02-cloud-native-e-automacao.md</code>	—	Atendido
Demonstração do sistema rodando em containers	<code>compose ps</code> healthy + Swagger	<code>10-docker-compose-ps.png</code> , <code>11-docker-desktop-container.png</code> , <code>13-swagger.png</code>	Atendido
Relatório de vantagens/diferenças/persistência	Seções 9 e 10 deste documento	—	Atendido
Monitoramento de CPU/memória/disco/logs	<code>./scripts/monitor_system.sh</code>	<code>18-monitor-system-sh.png</code>	Atendido
Backup de banco	<code>./scripts/backup_db.sh</code>	<code>16-backup-db-sh.png</code> , <code>17-</code>	Atendido

Exigência da Entrega 2	Artefato	Evidência (print)	Aderência
		backup-arquivo-gerado.png	
Gerenciamento de processos	<code>./scripts/manage_services.sh</code>	19-manage-services-sh.png	Atendido
Pipes / redirecionamento / variáveis / cron / permissões	Scripts + Seção 8	Visíveis em 18-monitor-system-sh.png (pipes/tee/redir), 19-manage-services-sh.png, prints do deploy.sh	Atendido

8. Conceitos Linux/Bash demonstrados

Conceito	Onde aparece nos scripts	Evidência
Variáveis	<code>DEPLOY_ENV</code> , <code>API_URL</code> , <code>BACKUP_RETENTION_DAYS</code> , <code>INTERVAL</code> , <code>DURATION</code> , <code>HEALTH_TIMEOUT</code>	Prints 16, 18, 20–24
Condicionais	<code>if</code> , <code>case</code> em <code>manage_services.sh</code> e <code>deploy.sh</code>	Prints 19, 24
Funções	<code>check_health</code> , <code>save_current_images</code> , <code>rollback</code> , <code>restart_with_healthcheck</code>	Prints 20–24
Pipes	<code>awk</code> , <code>grep</code> , <code>tee</code> , <code>paste -sd ';' </code> em <code>monitor_system.sh</code>	Print 18
Redirecionamento	<code>></code> , <code>>></code> , <code>2>&1</code> em todos os scripts	Prints 18, 19, 23
Logs	<code>logs/deploy.log</code> , <code>logs/monitor/metrics_*.log</code>	Print 18 (mostra o caminho do log gerado)
Permissões	<code>chmod 644</code> no log final do <code>monitor_system.sh</code> ; <code>chmod +x</code> nos scripts	Print 18
Cron	Sugestões impressas pelo <code>monitor_system.sh</code> ao final e documentadas em <code>docs/02-cloud-native-e-automacao.md</code>	Print 18

Exemplos de cron documentados

```
0 2 * * * cd /path/to/maya-rpg-api && ./scripts/backup_db.sh >> /var/log/maya-rpg/backup.log 2>&1

*/5 * * * * cd /path/to/maya-rpg-api && ./scripts/monitor_system.sh 5 30 >> /var/log/maya-rpg/monitor.log 2>&1
```

9. Ambiente tradicional vs. containerizado

Critério	Ambiente tradicional	Ambiente containerizado
Instalação	Node, PostgreSQL e dependências instalados manualmente	Imagens Docker definem o ambiente
Reprodutibilidade	Varia entre máquinas/SOs	Mesmo Compose sobre os mesmos serviços
Configuração	Depende da máquina local	Centralizada em <code>.env</code> e Compose
Isolamento	Processos compartilham o SO host	Containers isolados em rede dedicada (<code>maya-network</code>)
Persistência	Depende da instalação local do banco	Volume nomeado <code>pg_data</code> + bind mounts
Deploy	Passos manuais	<code>./scripts/deploy.sh</code> automatizado

Vantagens da containerização observadas nesta entrega

- ambiente reproduzível em qualquer máquina com Docker;
- isolamento claro entre API e banco;
- persistência explícita via volume nomeado;
- previsibilidade na demonstração acadêmica;
- menor acoplamento ao SO local (Windows neste caso, mas equivalente em Linux).

Estratégia de volumes/persistência

Volume/Pasta	Tipo	Função
<code>pg_data:/var/lib/postgresql/data</code>	Volume nomeado	Persistência do PostgreSQL entre recriações de container
<code>./uploads:/app/uploads</code>	Bind mount	Arquivos enviados pelo paciente/profissional
<code>./logs:/app/logs</code>	Bind mount	Logs gerados pela API
<code>./backups</code>	Diretório local	Dumps gerados pelo <code>backup_db.sh</code>

`down -v` é usado apenas quando se deseja descartar o volume do banco — caso contrário, `down` preserva os dados.

10. Como os scripts ajudam o ciclo de desenvolvimento

- `setup_env.sh` reduz erros de configuração no primeiro uso da máquina.
- `monitor_system.sh` documenta o uso de recursos durante a demonstração.
- `backup_db.sh` protege dados de desenvolvimento e demonstração.
- `manage_services.sh` padroniza start/stop/restart e oferece modo `watch` para resiliência.

- `deploy.sh` reúne build, backup, migração e healthcheck em um único fluxo auditável (com `logs/deploy.log`).

11. Teste de carga (k6) — resultado real

Validado em 11/05/2026 com `k6 run test/load/load-test.js`. Evidência: `25-k6-load-test.png`.

Métrica	Valor obtido	Threshold	Resultado
<code>http_req_duration</code> p(95)	2.42 ms	< 500 ms	✓ Aprovado
<code>http_req_failed</code> rate	0.00%	< 1%	✓ Aprovado
Checks succeeded	5451 / 5451 (100%)	—	✓
Iterations	1817 (15.07/s)	—	—
VUs (max)	20	—	—
Duração	2m30s (3 stages com graceful ramp-down)	—	—

Cenários validados pelo `load-test.js`:

- `login` responde 401 ou 429
- `check-in` responde 401, 403 ou 429
- `historico` responde 401, 403 ou 429

Nota: os 401/403 são esperados nesse cenário porque o teste de carga não envia token válido — o objetivo é medir o comportamento do gateway de autenticação sob carga, não autenticar de fato.

12. Evidências visuais — índice completo

Todos os prints estão em `imagens-maya/cloud-native/`. Ordem de leitura recomendada para o avaliador:

1. **Pré-condições (build local fora de container)** — `01-env-configurado.png`, `02-npm-lint.png`, `03-npm-build.png`, `04-npm-test-23-passing.png`.
2. **Validação do Compose** — `05-docker-compose-config-api.png`, `06-docker-compose-config-db.png`, `07-arquivos-docker.png`.
3. **Build e execução em container** — `08-docker-compose-build.png`, `09-docker-compose-up-healthy.png`, `10-docker-compose-ps.png`, `11-docker-desktop-container.png`, `12-docker-desktop-logs.png`.
4. **Aplicação respondendo** — `13-swagger.png`.
5. **Persistência e isolamento** — `14-volume-ls-pg-isready.png`, `15-uploads-logs.png`.
6. **Scripts de automação** — `16-backup-db-sh.png`, `17-backup-arquivo-gerado.png`, `18-monitor-system-sh.png`, `19-manage-services-sh.png`.

7. **Deploy automatizado** — `20-deploy-sh-inicio.png` → `24-deploy-sh-healthy.png` (5 prints sequenciais).
8. **Teste de carga** — `25-k6-load-test.png`.
-

13. Conclusão

A entrega de Cloud Native cobre todos os requisitos obrigatórios da Entrega 2 do PI (Dockerfile da API, Dockerfile do banco, `docker-compose.yml` com volume persistente, variáveis de ambiente, script de deploy automatizado, documentação e demonstração do sistema rodando em containers). Os 25 prints em `imagens-maya/cloud-native/` evidenciam execução real em 11/05/2026, incluindo o teste de carga com k6 (p95=2.42ms, 0% de falha em 5451 checks). A solução está pronta para demonstração acadêmica em ambiente local containerizado.

Entrega 2 — Programação Mobile e Cloud Native

Projeto: Maya Fisioterapia/RPG

Curso: Projeto Interdisciplinar — 3º ADS — FECAP 2026

Aluno (escopo individual desta entrega): Programação Mobile + Cloud Native (backend/API containerizada)

Validação técnica: 11/05/2026 — ambiente local containerizado

Evidências visuais: [imagens-maya/](#)

Este documento é o ponto de entrada da Entrega 2 para o avaliador. Apresenta de forma objetiva o que foi implementado, como executar e onde estão as evidências.

Para um roteiro visual completo com cada etapa seguida da sua captura de tela real, consulte **00-guia-ilustrado.md**.

1. Visão Geral

O projeto **Maya Fisioterapia/RPG** apoia o acompanhamento fisioterapêutico de pacientes da Clínica Maya Yoshiko Yamamoto por meio de aplicativo mobile, API backend, banco de dados e infraestrutura containerizada.

O escopo individual desta entrega cobre duas frentes:

Frente	Objetivo
Programação Mobile	App Android do paciente — autenticação, plano de exercícios, check-in com dor 0–10, histórico/evolução, persistência local (Room/SQLite) e consumo da API.
Cloud Native	API NestJS + PostgreSQL containerizados com Docker Compose, volume persistente, variáveis de ambiente, scripts Bash/Linux (setup, monitoramento, backup, gerenciamento, deploy) e teste de carga com k6.

O backend (NestJS), o módulo web e a parte de testes/qualidade são entregas integradas do projeto. O foco deste documento é o que o aluno entrega individualmente nas disciplinas de Programação Mobile e Cloud Native — partes do grupo aparecem apenas como contexto quando são dependência direta.

2. Escopo e Estrutura

```
Projeto3/
├── Documentos/Entrega2/

|   ├── ProgramacaoMobile/          # Documentação do app Android

|   └── Sistemas Operacionais e Arquiteturas Cloud Native/

|       ├── Dockerfile              # API NestJS (multi-stage, Node 20-alpine)

|       ├── Dockerfile.db           # PostgreSQL 16-alpine

|       ├── docker-compose.yml      # api + db + rede + volume

|       ├── .env.example             # Modelo de variáveis de ambiente

|       ├── scripts/                # setup, monitor, backup, manage, deploy

|       └── RELATORIO_CLOUD_NATIVE.md

|           └── docs/                # Este pacote de documentação

└── src/Entrega 2/

    ├── maya-rpg-api/               # Código do backend (espelho do que está em Cloud Native)

    └── mobile/                     # Código do app Android
```

E, fora do repositório acadêmico, a pasta de evidências:

```
Imagens/
├── cloud-native/    # 25 prints validados em 11/05/2026

├── mobile/          # 17 prints validados em 11/05/2026

└── postman-api/     # 12 prints validados em 11/05/2026
```

3. Responsabilidades Documentadas

3.1 Programação Mobile (app Android — paciente)

Requisito do PI (Entrega 2)	Implementação	Status	Evidência
Consumir API REST	Retrofit 2 + Gson	OK	README mobile, build sucesso
Tratamento de JSON	Gson	OK	README mobile
Armazenamento local com SQLite	Room (tabela <code>exercise_sessions</code>)	OK	README mobile, VALIDACAO_FINAL
Autenticação	JWT (login por e-mail ou CPF)	OK	README mobile
Check-in de exercício	<code>exerciseId</code> , <code>painLevel</code> 0–10, <code>notes</code> , <code>executedAt</code>	OK	README mobile
Nível de dor 0–10	Slider Material	OK	README mobile
Observações no check-in	Campo de texto	OK	README mobile
Histórico/evolução	Tela de evolução com MPAndroidChart	OK	README mobile
Sincronização offline	WorkManager (<code>SyncWorker</code>)	OK	README mobile
Notificações/lembretes	FCM + <code>ReminderWorker</code> integrados no código	Parcial	Implementação presente, sem print de notificação real no dispositivo até esta versão da entrega
Aceite LGPD	<code>LgpdConsentActivity</code> + endpoint <code>/api/auth/accept-lgpd</code>	OK	Código mobile + Swagger (print 13)
Múltiplas Activities + Intents	Estrutura de navegação	OK	README mobile
Fragment real	<code>ExercisePlanActivity</code>	OK	README mobile
ConstraintLayout em todos os layouts	XMLs padronizados	OK	README mobile
TextView, ImageView, Button	Componentes presentes	OK	README mobile
Identidade visual da Clínica	Cores e logo aplicados	OK	README mobile

Validação fora do sandbox em 10/05/2026: `gradlew.bat :app:testDebugUnitTest` e `gradlew.bat :app:assembleDebug` finalizaram com código 0.

3.2 Cloud Native (backend containerizado)

Requisito do PI (Entrega 2)	Artefato (caminho real)	Print de evidência
Dockerfile do backend	<code>../Dockerfile</code>	<code>imagens-maya/cloud-native/07-arquivos-docker.png</code> , <code>08-docker-compose-build.png</code>
Dockerfile do banco	<code>../Dockerfile.db</code>	<code>06-docker-compose-config-db.png</code> , <code>08-docker-compose-build.png</code>
Docker Compose com API + banco	<code>../docker-compose.yml</code>	<code>05-docker-compose-config-api.png</code> , <code>06-docker-compose-config-db.png</code> , <code>09-docker-compose-up-healthy.png</code> , <code>10-docker-compose-ps.png</code>
Volume persistente	<code>pg_data</code> (nomeado)	<code>14-volume-ls-pg-isready.png</code>
Variáveis de ambiente	<code>../.env.example</code> + <code>environment:</code> no Compose	<code>01-env-configurado.png</code>
Script de deploy automatizado	<code>../scripts/deploy.sh</code>	<code>20-deploy-sh-inicio.png</code> → <code>24-deploy-sh-healthy.png</code>
Demonstração do sistema rodando em containers	<code>compose ps</code> , Docker Desktop, Swagger	<code>10-docker-compose-ps.png</code> , <code>11-docker-desktop-container.png</code> , <code>12-docker-desktop-logs.png</code> , <code>13-swagger.png</code>
Documentação de build/execução	Este doc + <code>02-cloud-native-e-automacao.md</code>	—
Relatório (vantagens, ambiente tradicional × containerizado, volumes/persistência)	<code>../RELATORIO_CLOUD_NATIVE.md</code>	—
Script de monitoramento (CPU/memória/disco/logs)	<code>../scripts/monitor_system.sh</code>	<code>18-monitor-system-sh.png</code>
Script de backup	<code>../scripts/backup_db.sh</code>	<code>16-backup-db-sh.png</code> , <code>17-backup-arquivo-gerado.png</code>
Gerenciamento de processos	<code>../scripts/manage_services.sh</code>	<code>19-manage-services-sh.png</code>

Requisito do PI (Entrega 2)	Artefato (caminho real)	Print de evidência
Pipes/redirect/variáveis/cron/permissões	Scripts + Seção 8 do Relatório	Visíveis em prints 18, 19, 20–24

Testes, Qualidade e Evidências da Entrega

Projeto: Maya Fisioterapia/RPG

Objetivo: apresentar comandos de validação, testes automatizados, teste de carga, prints e critérios de qualidade da Entrega 2.

Validação técnica: 11/05/2026 — ambiente local containerizado

Evidências: `imagens-maya/`

1. Objetivo do Documento

Reúne, num único lugar, tudo o que o avaliador precisa para verificar a entrega:

- comandos de instalação, build, lint;
- testes unitários e e2e;
- teste de carga com k6 (resultado real);
- teste de sistema/aceitação;
- critérios de qualidade conforme ISO/IEC 25010;
- índice de prints já anexados e lista do que ainda falta capturar.

2. Validação Rápida

Backend (na pasta com `package.json`):

```
npm install
npm run lint

npm run build

npm test -- --runInBand

npm run test:e2e
```


Backend via Docker:

```
docker compose config
docker compose up --build -d

docker compose ps
```

Teste de carga:

```
k6 run test/load/load-test.js
```

Mobile (em `src/Entrega 2/mobile/`):

```
./gradlew :app:assembleDebug
```

Windows:

```
.\gradlew.bat :app:assembleDebug
```

3. Testes Unitários — resultado real

Arquivo	Finalidade
<code>src/auth/auth.service.spec.ts</code>	Regras de autenticação
<code>src/patients/patients.service.spec.ts</code>	Regras de pacientes
<code>src/check-ins/check-ins.service.spec.ts</code>	Regras de check-in
<code>src/dashboard/dashboard.service.spec.ts</code>	Indicadores de dashboard
<code>src/common/lgpd/lgpd.service.spec.ts</code>	Funções de LGPD
<code>src/app.controller.spec.ts</code>	Controller raiz

Comando:

```
npm test -- --runInBand
```

Resultado obtido em 11/05/2026 (print `imagens-maya/cloud-native/04-npm-test-23-passing.png`):

```
Test Suites: 6 passed, 6 total
Tests:      23 passed, 23 total
```

Snapshots: 0 total

Time: 3.028 s

4. Testes de Integração / E2E

Arquivo	Finalidade
test/app.e2e-spec.ts	Inicialização da aplicação e endpoint base
test/dashboard.e2e-spec.ts	Endpoints de dashboard
test/acceptance.e2e-spec.ts	Fluxo de aceitação

Comando:

```
npm run test:e2e
```

Print recomendado: terminal com suítes passando, sem erros de conexão com o banco e tempo total visível. Pendente nesta entrega — usar o mesmo padrão do print `04-npm-test-23-passing.png`.

5. Teste de Carga com k6 — resultado real

Arquivo: `test/load/load-test.js`. Comando:

```
k6 run test/load/load-test.js
```

Configuração:

- 3 stages com graceful ramp-down;
- até 20 VUs;
- duração $\approx$ 2m30s;
- thresholds:

- `http_req_duration: ['p(95)<500']`

- `http_req_failed: ['rate<0.01']`

Resultado obtido em 11/05/2026 (print `imagens-maya/cloud-native/25-k6-load-test.png`):

Métrica	Valor	Threshold	Resultado
<code>http_req_duration</code> p(95)	2.42 ms	< 500 ms	

Métrica	Valor	Threshold	Resultado
<code>http_req_failed</code>	0.00%	<code>< 1%</code>	
Checks succeeded	5451 / 5451 (100%)	—	
Iterations	1817 (15.07/s)	—	—
VUs máx	20	—	—
Data received	6.5 MB (54 kB/s)	—	—
Data sent	1.2 MB (10 kB/s)	—	—

Cenários validados:

- `login responde 401 ou 429`
- `check-in responde 401, 403 ou 429`
- `historico responde 401, 403 ou 429`

Os retornos 401/403 são esperados porque o teste de carga não envia token válido — o objetivo é medir o comportamento do gateway de autenticação sob carga.

Relatório do teste de carga

Campo	Valor
Data da execução	11/05/2026
Ambiente	Local — Windows 11 + Docker Desktop (API e DB containerizados)
Comando	<code>k6 run test/load/load-test.js</code>
Usuários virtuais (máx)	20
Duração	2m30s (3 stages)
p(95)	2.42 ms
Taxa de falha	0.00%
Resultado	Aprovado em ambos os thresholds
Observações	Cenários validam o comportamento do gateway de autenticação sob carga (respostas 401/403/429 esperadas)

6. Teste de Sistema / Aceitação

Arquivo: `test/acceptance.e2e-spec.ts`. Objetivos:

- validar fluxo integrado da API;
- verificar funcionamento conjunto das funcionalidades principais;
- gerar evidência de comportamento do sistema como um todo.

Fluxos demonstráveis no Swagger ou via Postman:

Fluxo	Endpoint(s)	Evidência
Login	<code>POST /api/auth/login</code>	Print Swagger ou Postman retornando token JWT
Consulta de exercícios	<code>GET /api/exercises</code>	Print retornando lista
Check-in	<code>POST /api/check-ins</code>	Print do registro criado (status 201)
Histórico	<code>GET /api/check-ins/me</code> (ou similar)	Print retornando registros
Dashboard	<code>GET /api/dashboard/...</code>	Print dos indicadores

Status atual: implementado no código (`test/acceptance.e2e-spec.ts`). Prints via Postman/Swagger são pendentes (Seção 9.3).

7. Qualidade de Software — ISO/IEC 25010

Característica	Aplicação no projeto	Evidência
Adequação funcional	Endpoints e telas cobrem autenticação, pacientes, prescrições, check-ins, histórico e indicadores	<code>13-swagger.png</code> (Swagger expando todos os grupos)
Eficiência de desempenho	Paginação, teste de carga com thresholds (p95 < 500ms)	<code>25-k6-load-test.png</code> (p95 = 2.42 ms)
Compatibilidade	API REST consumida por mobile e web	README mobile, <code>13-swagger.png</code>
Usabilidade	Swagger documenta a API; mobile tem fluxo direto para o paciente	<code>13-swagger.png</code> , README mobile
Confiabilidade	Healthchecks, Docker Compose, scripts de restart, testes automatizados	<code>09-</code> , <code>10-</code> , <code>19-manage-services-sh.png</code> , <code>04-npm-test-23-passing.png</code>
Segurança	JWT, bcrypt, guards/roles, LGPD, <code>.env</code> fora do Git	Código backend, <code>01-env-configurado.png</code>
Manutenibilidade	Organização por módulos NestJS, testes, lint	<code>02-npm-lint.png</code> , <code>03-npm-build.png</code>
Portabilidade	Dockerfile e Compose permitem execução em diferentes máquinas	Prints 05–10

Processo de qualidade aplicado:

Requisitos → Implementação → Lint → Build → Testes → Docker → Carga → Evidências → Entrega

8. Guia de Prints — Backend / Cloud Native

Já anexados em [imagens-maya/cloud-native/](#) (25 prints validados em 11/05/2026):

Nº	Evidência	Print
1	<code>.env</code> configurado	<code>01-env-configurado.png</code>
2	Lint sem erro	<code>02-npm-lint.png</code>
3	Build concluído	<code>03-npm-build.png</code>
4	Testes unitários passando (23/23)	<code>04-npm-test-23-passing.png</code>
5	<code>docker compose config</code> (api)	<code>05-docker-compose-config-api.png</code>
6	<code>docker compose config</code> (db)	<code>06-docker-compose-config-db.png</code>
7	Arquivos Docker no diretório	<code>07-arquivos-docker.png</code>
8	<code>docker compose build</code>	<code>08-docker-compose-build.png</code>
9	<code>docker compose up -d</code> healthy	<code>09-docker-compose-up-healthy.png</code>
10	<code>docker compose ps</code> healthy	<code>10-docker-compose-ps.png</code>
11	Docker Desktop — container ativo	<code>11-docker-desktop-container.png</code>
12	Docker Desktop — logs da API	<code>12-docker-desktop-logs.png</code>
13	Swagger aberto	<code>13-swagger.png</code>
14	Volume <code>pg_data</code> + <code>pg_isready</code>	<code>14-volume-ls-pg-isready.png</code>
15	Bind mounts <code>/app/uploads</code> e <code>/app/logs</code>	<code>15-uploads-logs.png</code>
16	<code>backup_db.sh</code> executado	<code>16-backup-db-sh.png</code>
17	Arquivo <code>.sql</code> em <code>backups/</code>	<code>17-backup-arquivo-gerado.png</code>
18	<code>monitor_system.sh</code> rodando	<code>18-monitor-system-sh.png</code>
19	<code>manage_services.sh</code> status + restart	<code>19-manage-services-sh.png</code>
20–24	<code>deploy.sh</code> (5 etapas)	<code>20-</code> a <code>24-deploy-sh-*.png</code>
25	k6 load test	<code>25-k6-load-test.png</code>

9. Índice de prints anexados

9.1 Backend / API — opcional

Print sugerido	Comando para gerar
Testes e2e passando	<code>npm run test:e2e</code> (print do terminal com suites passando)

9.2 Mobile — [Imagens/mobile/](#) — 17 prints

#	Arquivo	Evidência
1	01-build-android-studio-successful.png	BUILD SUCCESSFUL — assembleDebug
2	02-splash-bem-vindo.png	Splash screen
3	03-login-vazio.png	Tela de login (campos vazios)
4	04-login-preenchido.png	Tela de login preenchida
5	05-primeiro-acesso-criar-senha.png	Primeiro Acesso — criar senha
6	06-lgpd-termo-aceitar.png	Termo LGPD — aguardando aceite
7	07-lgpd-salvando.png	Termo LGPD — aceito, salvando
8	08-fcm-permission-dialog.png	Diálogo FCM de permissão (Android nativo)
9	09-home-dashboard.png	Home — dashboard do paciente
10	10-minha-evolucao-grafico.png	Minha Evolução — gráfico de dor
11	11-plano-exercicios-lista.png	Plano de Exercícios — lista
12	12-configuracoes.png	Configurações
13	13-configuracoes-sair-confirmacao.png	Confirmação de logout
14	14-editar-perfil.png	Editar Perfil
15	15-agendar-sessao-calendario.png	Agendar Sessão — calendário
16	16-mensagens.png	Mensagens
17	17-minha-agenda.png	Minha Agenda

9.3 Postman / API client — [Imagens/postman-api/](#) — 12 prints

#	Arquivo	Endpoint	Status
1	01-admin-login-201-token.png	POST /api/auth/login (admin)	201
2	02-auth-me-200-admin.png	GET /api/auth/me	200
3	03-post-patients-201.png	POST /api/patients	201
4	04-post-exercises-201.png	POST /api/exercises	201
5	05-patient-login-201-primeiro-acesso.png	POST /api/auth/login (paciente, CPF)	201
6	06-auth-change-password-201.png	POST /api/auth/change-password	201
7	07-patient-login-201-token.png	POST /api/auth/login (nova senha)	201
8	08-accept-lgpd-201.png	POST /api/auth/accept-lgpd	201
9	09-post-prescriptions-201.png	POST /api/prescriptions	201

#	Arquivo	Endpoint	Status
10	10-get-prescriptions-me-full-200.png	GET /api/prescriptions/me/full	200
11	11-post-check-ins-201.png	POST /api/check-ins	201
12	12-get-check-ins-history-200.png	GET /api/check-ins/my-history	200

10. Checklist Final Antes de Entregar

```
git status --short
```

Não enviar:

```
.env  
node_modules/  
  
dist/  
  
logs/  
  
uploads/  
  
backups/
```

Devem estar presentes:

```
Dockerfile  
Dockerfile.db  
  
docker-compose.yml  
  
.env.example  
  
scripts/*.sh  
  
scripts/migrations/*.sql  
  
RELATORIO_CLOUD_NATIVE.md
```

docs/00-guia-ilustrado.md

docs/01-entrega-2-mobile-e-cloud.md

docs/02-cloud-native-e-automacao.md

docs/03-testes-e-evidencias.md

Imagens/cloud-native/*.png (25 prints)

Imagens/mobile/*.png (17 prints)

Imagens/postman-api/*.png (12 prints)

Imagens/README.md

11. Organização dos Anexos

Imagens/

├─ cloud-native/ # 25 prints validados em 11/05/2026

| └─ 01-env-configurado.png

| └─ ...

| └─ 25-k6-load-test.png

├─ mobile/ # 17 prints validados em 11/05/2026

| └─ 01-build-android-studio-successful.png

| └─ ...

| └─ 17-minha-agenda.png

└─ postman-api/ # 12 prints validados em 11/05/2026

└─ 01-admin-login-201-token.png

└─ ...

└─ 12-get-check-ins-history-200.png

12. Conclusão

A entrega tem evidência real e mensurável em todas as três áreas. Cloud Native: 25 prints cobrindo build, lint, 23/23 testes unitários PASS, Compose configurado, containers healthy, volume persistente, scripts de automação executados e teste de carga com p95=2.42ms e 0% de falha. Mobile Android: 17 prints cobrindo o fluxo completo do paciente (splash → login → primeiro acesso → LGPD → FCM → home → exercícios → evolução → agenda → configurações). API via Postman: 12 prints cobrindo o fluxo completo (admin → criar paciente/exercício → login paciente → LGPD → prescrição → check-in → histórico). Total: 54 prints validados em 11/05/2026.

Evidências Visuais — Cloud Native (seleção)

```
.env
# --- Banco de dados (PostgreSQL) ---
DB_HOST=localhost
DB_PORT=5432
DB_USER=maya_user
DB_PASSWORD=maya_pass
DB_NAME=maya_rpg
DB_SSL=false

# --- JWT ---
JWT_SECRET=test_jwt_secret_123456789
JWT_EXPIRES_IN=15m

# --- CORS ---
# Lista separada por vírgula. Aceita "*" para liberar tudo (apenas em dev).
CORS_ORIGINS=http://localhost:4200,http://localhost:3000,https://maya-rpg-web.vercel.app

# --- Servidor ---
PORT=3000
NODE_ENV=development

# --- Arquivos publicos ---
API_URL=http://localhost:3000

# --- Dados de apresentacao ---
SEED_DEMO_DATA=false
SEED_ADMIN_DEFAULT=false
SEED_ADMIN_EMAIL=
SEED_ADMIN_PASSWORD=
SEED_ADMIN_NAME=Administrador Maya

# --- E-mail (SMTP) ---
# Em desenvolvimento, se nao configurado, usa Ethereum Email automaticamente.
SMTP_HOST=smtp.ethereal.email
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
MAIL_FROM=noreply@maya-rpg.com.br

# --- Firebase Cloud Messaging (notificacoes push) ---
FCM_SERVER_KEY=

# --- Mercado Pago ---
MP_ACCESS_TOKEN=TEST-xxxxx-xxxxx-xxxxx
MP_WEBHOOK_SECRET=xxxxx-xxxxx-xxxxx-xxxxx
MP_NOTIFICATION_URL=https://seu-dominio.com/api/webhooks/mercadopago

# --- Deploy ---
DEPLOY_ENV=staging
DEPLOY_URL=https://seu-dominio.com
```

1. .env configurado com todas as variáveis de ambiente

```
PS C:\Users\neiso\OneDrive\Workspaces\maya-rpg-api> npm run lint
> maya-rpg-api@0.0.1 lint
> eslint "{src,apps,libs,test}/**/*.ts" --fix

PS C:\Users\neiso\OneDrive\Workspaces\maya-rpg-api> npm run build
> maya-rpg-api@0.0.1 build
> nest build

PS C:\Users\neiso\OneDrive\Workspaces\maya-rpg-api> npm test -- --runInBand
> maya-rpg-api@0.0.1 test
> jest --runInBand

PASS src/check-ins/check-ins.service.spec.ts
PASS src/auth/auth.service.spec.ts
PASS src/common/jgpd/jgpd.service.spec.ts
PASS src/dashboard/dashboard.service.spec.ts
PASS src/patients/patients.service.spec.ts
PASS src/app.controller.spec.ts

Test Suites: 6 passed, 6 total
Tests: 23 passed, 23 total
Snapshots: 0 total
Time: 3.028 s, estimated 4 s
Ran all test suites.
PS C:\Users\neiso\OneDrive\Workspaces\maya-rpg-api>
```

4. Testes unitários: 6 suites, 23/23 PASS em 3.028s

```
Problems Output Debug Console Terminal Ports
PS C:\Users\nelso\OneDrive\Nelso\Desktop\Workspace\maya-rpg-api> dir Dockerfile, Dockerfile.db, docker-compose.yml

Diretório: C:\Users\nelso\OneDrive\Nelso\Desktop\Workspace\maya-rpg-api

Mode                LastWriteTime         Length Name
----                -
-a---1             09/05/2026      28:02           786 Dockerfile
-a---1             06/05/2026      17:24           123 Dockerfile.db
-a---1             11/05/2026      12:18          1586 docker-compose.yml

PS C:\Users\nelso\OneDrive\Nelso\Desktop\Workspace\maya-rpg-api> |
```

7. Artefatos Docker no diretório (Dockerfile, Dockerfile.db, docker-compose.yml)

```
Problems Output Debug Console Terminal Ports
PS C:\Users\nelso\OneDrive\Nelso\Desktop\Workspace\maya-rpg-api> docker compose build

#18 CACHED

#19 [api stage-1 4/8] COPY package*.json ./
#19 CACHED

#20 [api stage-1 5/8] RUN npm ci --omit-dev
#20 CACHED

#21 [api stage-1 6/8] COPY --from=builder /app/dist ./dist
#21 DONE 0.1s

#22 [api stage-1 7/8] RUN mkdir -p /app/uploads/app/logs && chown -R appuser:appgroup /app/uploads/app/logs
#22 DONE 0.2s

#23 [api stage-1 8/8] RUN apk add --no-cache curl
#23 0.712 ( 1/10) Installing brotli-libs (1.2.0-r0)
#23 0.749 ( 2/10) Installing c-ares (1.34.6-r0)
#23 0.763 ( 3/10) Installing libunistring (1.4.1-r0)
#23 0.826 ( 4/10) Installing libidn2 (2.3.8-r0)
#23 0.866 ( 5/10) Installing nghttp2-libs (1.60.0-r0)
#23 0.876 ( 6/10) Installing nghttp3 (1.13.1-r0)
#23 0.884 ( 7/10) Installing libpsl (0.21.5-r3)
#23 0.893 ( 8/10) Installing zstd-libs (1.5.7-r2)
#23 0.924 ( 9/10) Installing libcurl (8.17.0-r1)
#23 0.953 (10/10) Installing curl (8.17.0-r1)
#23 0.971 Executing busybox-1.37.0-r30.trigger
#23 0.978 OK: 16.0 MiB in 28 packages
#23 DONE 1.0s

#24 [api] exporting to image
#24 exporting layers
#24 exporting layers 0.2s done
#24 exporting manifest sha256:ff10dc3bd86d72a7b9ae82c6c0b01b1f2c04d05e403ffdb8e3c345fcbh 0.0s done
#24 exporting config sha256:4e81b0d531399f5f6a5c1179a2b79903c09a1f0b6d8f37f5a86979a25eaf762 0.0s done
#24 exporting attestation manifest sha256:014fe170f57a1c4a5a62e1f9e645b140bf05816fa3c095524846f0680961f780 0.0s done
#24 exporting manifest list sha256:2db8f3084942d0dd09f74cf0cf25deb651b874bd0793e3a7f0e5953f709e876a3 0.0s done
#24 naming to docker.io/library/maya-rpg-api-api:latest done
#24 unpacking to docker.io/library/maya-rpg-api-api:latest 0.1s done
#24 DONE 0.5s

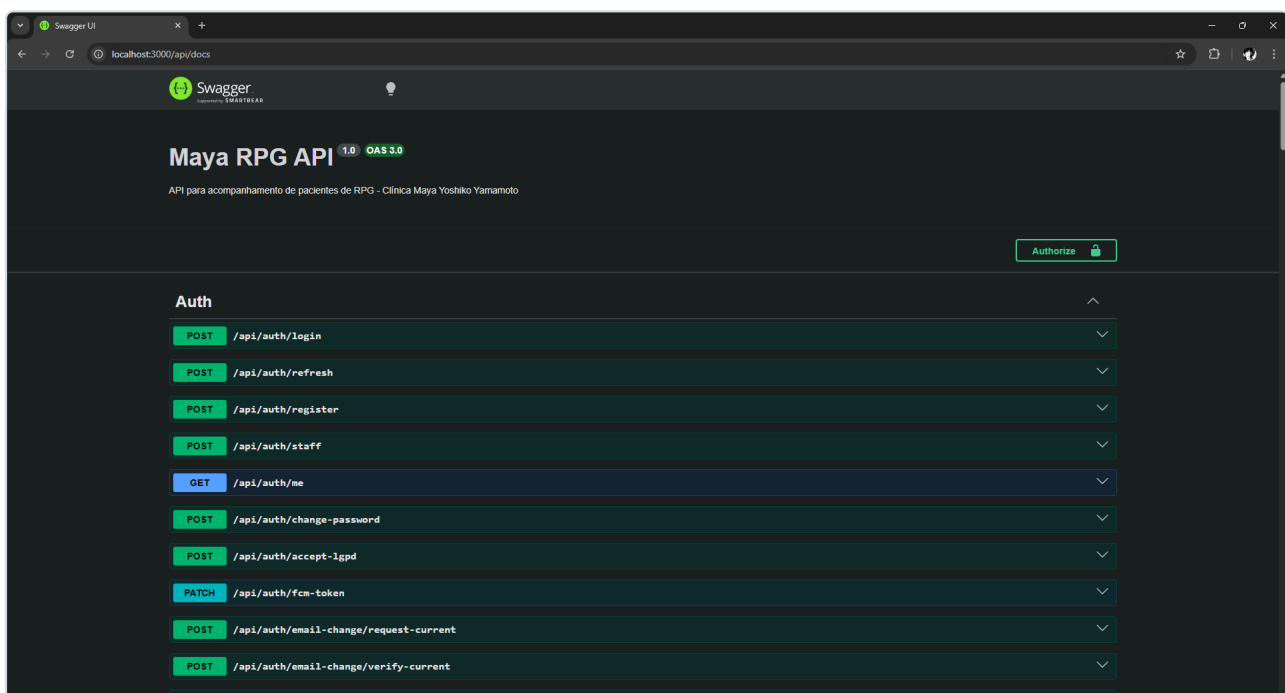
#25 [api] resolving provenance for metadata file
#25 DONE 0.0s
[+] build 2/2
  ✓ Image maya-rpg-db:latest Built 10.0s
  ✓ Image maya-rpg-api-api Built 10.0s
● PS C:\Users\nelso\OneDrive\Nelso\Desktop\Workspace\maya-rpg-api> docker compose up -d
[+] up 2/2
  ✓ Container maya-rpg-db Healthy 12.1s
  ✓ Container maya-rpg-api Started 11.9s

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs
PS C:\Users\nelso\OneDrive\Nelso\Desktop\Workspace\maya-rpg-api> |
```

9. docker compose up --build -d → containers Healthy

```
PS C:\Users\neiso\OneDrive\Nelson\Desktop\Workspace\maya-rpg-api> docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS                PORTS
maya-rpg-api         maya-rpg-api-api                  "docker-entrypoint.s..."          api        35 seconds ago Up 23 seconds (healthy) 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
maya-rpg-db          maya-rpg-db:latest                "docker-entrypoint.s..."          db         36 seconds ago Up 34 seconds (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
```

10. docker compose ps → API e DB Up (healthy)



13. Swagger UI em <http://localhost:3000/api/docs>

```
Problems Output Debug Console Terminal Ports
PS C:\Users\nelson\OneDrive\Nelson\Desktop\Workspace\maya-rpg-api> docker volume ls
DRIVER      VOLUME NAME
local       maya-rpg-api_pg_data
PS C:\Users\nelson\OneDrive\Nelson\Desktop\Workspace\maya-rpg-api> docker compose exec db pg_isready -U maya_user -d maya_rpg
/var/run/postgresql:5432 - accepting connections
PS C:\Users\nelson\OneDrive\Nelson\Desktop\Workspace\maya-rpg-api>
```

14. Volume pg_data + pg_isready accepting connections

```
Problems Output Debug Console Terminal Ports
nelson@DESKTOP_MINGW64 ~/OneDrive/Nelson/Desktop/Workspace/maya-rpg-api (cleanup/entrega-2)
$ ./scripts/monitor_system.sh 5 15
=====
Monitoramento Maya RPG - Mon, May 11, 2026 12:54:56 PM
Intervalo: 5s | Duração: 15s
=====
[2026-05-11 12:54:56] CPU=14% | MEM=N/A/N/AMB (N/A%) | DISK=41% (Livre: 5586) | Containers: maya-rpg-api: Up 4 minutes (healthy);maya-rpg-db: Up 4 minutes (healthy)
[2026-05-11 12:55:03] CPU=2% | MEM=N/A/N/AMB (N/A%) | DISK=41% (Livre: 5586) | Containers: maya-rpg-api: Up 4 minutes (healthy);maya-rpg-db: Up 4 minutes (healthy)
[2026-05-11 12:55:09] CPU=2% | MEM=N/A/N/AMB (N/A%) | DISK=41% (Livre: 5586) | Containers: maya-rpg-api: Up 4 minutes (healthy);maya-rpg-db: Up 4 minutes (healthy)
=====
Monitoramento concluído em Mon, May 11, 2026 12:55:16 PM
Log salvo em: /c/Users/nelson/OneDrive/Nelson/Desktop/Workspace/maya-rpg-api/logs/monitor/metrics_20260511_125456.log

Dica: Para executar via cron a cada 5 minutos, adicione a linha:
*/5 * * * * ./scripts/monitor_system.sh 5 30 >> /c/Users/nelson/OneDrive/Nelson/Desktop/Workspace/maya-rpg-api/logs/monitor/cron_monitor.log 2>&1
Use 'crontab -e' para editar a crontab do usuário.

nelson@DESKTOP_MINGW64 ~/OneDrive/Nelson/Desktop/Workspace/maya-rpg-api (cleanup/entrega-2)
$ dir logs
monitor

nelson@DESKTOP_MINGW64 ~/OneDrive/Nelson/Desktop/Workspace/maya-rpg-api (cleanup/entrega-2)
$
```

18. monitor_system.sh — métricas CPU/memória/disco/containers

